

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 4: Building a RESTful API with Node.js and Express

CO/PO	CO2 / PO3, PO5
Objective	To design and implement a RESTful backend server with complete CRUD endpoints using Express middleware pipeline.
Prerequisites	- Basic JavaScript (functions, objects, arrays, async basics) - Understanding of HTTP methods (GET, POST, PUT, DELETE) and status codes
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 4 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 1 (RESTful API design with Node.js and Express, middleware pipeline, request lifecycle).
Architecture / Diagram	
Diagram	<pre> Client (Postman/Browser) v [Logging Middleware] v Express Router ├── GET /tasks → getAllTasks ├── POST /tasks → createTask ├── PUT /tasks/:id → updateTask ├── DELETE /tasks/:id → deleteTask v [Global Error Handler] </pre>
Problem Definition	
Problem Statement	- Build a Node/Express backend for a Task Management system. - Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet). - Apply a request logging middleware that logs method, URL, and timestamp for every incoming request. - Apply a global error handling middleware as the last middleware in the pipeline. - Use correct HTTP status codes (200, 201, 404, 500) for each response.
Key Questions / Analysis	- Why must the error handling middleware be defined last in the middleware chain? - What is the difference between <code>app.use()</code> and a route-specific middleware? - Why is it considered bad practice to send raw error stack traces to the client?
Supplementary Problems	- Add a middleware that rejects requests without a Content-Type: application/json header on POST/PUT. - Add a route-specific middleware that validates the task ID format before reaching the controller. - Add a 404 handler for undefined routes that returns a structured JSON response.

Key Skills Addressed	• Setting up an Express server and defining RESTful routes • Writing and applying custom middleware functions • Structuring a request-response-error pipeline correctly • Returning appropriate HTTP status codes for each operation
Learning Outcome	Students will be able to design and implement a complete RESTful API using Express, including a working middleware pipeline with logging and centralized error handling.
Lab Session Step-by-Step	
Steps	1. Initialize a new Node project and install Express: <pre>mkdir task-manager-api && cd task-manager-api npm init -y npm install express</pre> 2. Create server.js and set up a basic Express app: <pre>const express = require('express'); const app = express(); app.use(express.json()); app.listen(5000, () => console.log('Server running on port 5000'));</pre> 3. Add a request logging middleware applied globally: <pre>app.use((req, res, next) => { console.log(`\${req.method} \${req.url} - \${new Date().toISOString()}`); next(); });</pre> 4. Implement the 4 CRUD routes using an in-memory array as temporary storage. 5. Add a global error handling middleware as the final middleware: <pre>app.use((err, req, res, next) => { console.error(err.stack); res.status(500).json({ error: 'Something went wrong' }); });</pre> 6. Test all 4 endpoints using Postman or Thunder Client, verifying correct status codes.
Tools / Technology / Resources Required	• Node.js (v18+), npm • Express.js • Postman or Thunder Client • Visual Studio Code
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Demonstrate the request lifecycle on the whiteboard/diagram before coding students often treat middleware as 'magic'. • Emphasize calling next() in every middleware that doesn't end the response, or requests will hang indefinitely. • Show a live example of the error handler catching a deliberately thrown error. • Walk through Postman setup once as a class before students work independently.
Common Mistakes	• Forgetting to call next() inside custom middleware, causing the request to hang. • Placing the error handling middleware before the routes instead of after all of them. • Not setting app.use(express.json()), causing req.body to be undefined on POST/PUT. • Returning inconsistent response formats across different endpoints.

Troubleshooting Guide		
Symptom	Likely Cause	Fix
Request hangs indefinitely	Middleware does not call next() or send a response	Ensure every middleware either calls next() or ends the response with res.send/json
req.body is undefined	Missing express.json() middleware	Add app.use(express.json()) before route definitions
Error handler never triggers	Error handler defined before routes, or errors not passed via next(err)	Move error handler to be the last app.use(); call next(err) inside try/catch blocks
Postman shows 'Cannot GET /tasks'	Route path mismatch or server not restarted after changes	Verify route path spelling and restart the Node server
Student Assignment / Post Laboratory Work		
Tasks	- Review the Theory Concepts section and the assigned Coursera module before next week. - Add a middleware that rejects POST/PUT requests missing a Content-Type header. - Add a custom 404 handler for undefined routes.	
	GitHub Deliverables	
Expected Repository Contents	- New repository named task-manager-api-<rollno> - Working Express server with all 4 CRUD routes and middleware pipeline - Clean .gitignore (node_modules excluded) - At least one meaningful commit message describing the work	
Rubrics (Practical Evaluation / Viva)		
Criteria (Marks)		Description
Conceptual understanding (2 Marks)		Explains what and why, not just what was done
Implementation (4 Marks)		Code written, runs, and is original
Correctness (2 Marks)		Output matches expected result
Reflection (1 marks)		Errors documented and explained
Lab file evidence (1 marks)		Completeness and presentation
Total: 10 marks		Passing Threshold: 5 / 10 (50%)